Web Applications
Workshop 4

Semester II 2026
Sep 18, 2026
Prof. Francisco Hidrobo

ORM with FastAPI, SQLAlchemy, and MySQL

Report Content

1. General Information

Include:

1. Student name(s)
2. Course
3. Workshop number and title
4. Date
5. Computer/OS used
6. Python version
7. MySQL version
8. SQLAlchemy version
9. FastAPI version

2. Objective

In your own words, briefly explain the objective of the workshop.

Your explanation should address how **Object-Relational Mapping (ORM)** allows a Python application to interact with a relational database using objects instead of directly writing SQL statements.

Also briefly describe the architecture implemented:

```
Python Client / curl
      |
      | HTTP / JSON
      v
   FastAPI
      |
      | SQLAlchemy ORM
      v
    MySQL
```

The implemented application uses four related entities: `Customer`, `Order`, `OrderItem`, and `Product`.

3. Database and ORM Model

Present the database model created during the workshop.

Include a diagram showing the relationships:

```
Customer
  |
  +----< Order
         |
         +----< OrderItem >---- Product
```

For each entity, briefly describe its purpose, primary key, important attributes, and relationships with other entities.

Explain the purpose of `relationship()`, `ForeignKey`, `back_populates`, and `cascade="all, delete-orphan"` in the implemented SQLAlchemy models. The report should specifically identify which relationships use cascade behavior.

Evidence: Include a screenshot showing the four tables created in MySQL using:

SHOW **TABLES**;
The expected tables are `customers`, `orders`, `order_items`, and `products`.

4. CRUD Operations

Describe and demonstrate the CRUD operations implemented for **Customer** and **Product**.

Prepare a table similar to:

Operation	HTTP method	Endpoint	ORM operation
Create Customer	POST	/customers	db.add()
List Customers	GET	/customers	select()
Find Customer	GET	/customers/{id}	db.get()
Update Customer	PUT	/customers/{id}	Modify object + commit()
Delete Customer	DELETE	/customers/{id}	db.delete()

Create a similar description for Product.

Evidence: Include at least **two screenshots** demonstrating CRUD operations. At least one must show a successful creation or update and one must show the corresponding information stored in MySQL.

Explain why the application does not explicitly execute SQL statements such as:

INSERT INTO ...
UPDATE ...
DELETE FROM ...
even though these operations occur in the database. Relate your explanation to SQLAlchemy ORM and the SQL generated when `echo=True` is enabled.

5. Relationships and Nested Insert

Explain how an `Order` containing several `OrderItem` objects was created.

Use the workshop example conceptually:

```
{
  "customer_id": 1,
  "items": [
    {"product_id": 1, "quantity": 2},
    {"product_id": 2, "quantity": 1},
    {"product_id": 3, "quantity": 3}
  ]
}
```

Explain what happens when SQLAlchemy executes operations such as:

```
order = models.Order(customer=customer)
order.items.append(item)
db.add(order)
db.commit()
```

In particular, explain how the ORM manages the foreign-key relationships between Customer, Order, OrderItem, and Product.

Evidence: Include a screenshot of the corresponding records in both:

```
SELECT * FROM orders;
SELECT * FROM order_items;
```

6. Nested Queries

Demonstrate and explain the nested queries implemented in the workshop:

```
GET /orders/{order_id}
```

```
GET /customers/{customer_id}/orders
```

Include an example of the JSON returned by one of these endpoints and identify the nested objects.

For example, identify the hierarchy:

```
Customer
  └── Order
      └── OrderItem
          └── Product
```

Explain why ORM relationships make it possible to navigate this object structure. The workshop specifically defines response schemas that represent nested Customer → Order → OrderItem → Product information.

7. Update and Cascade Delete

Describe the experiment in which the status of an Order was updated.

Identify the generated SQL UPDATE observed in the Uvicorn/SQLAlchemy output and explain the relationship between changing:

```
order.status = data.status
```

and the SQL statement eventually executed in MySQL.

Next, explain the cascade-delete experiment.

Describe what happened to:

10. the Order;
11. its associated OrderItem objects;
12. the associated Product objects.

Explain **why the OrderItems disappeared but the Products remained**.

Include before-and-after evidence from the database. The workshop expects deleting an Order to remove its associated OrderItems while retaining Products.

8. Transactions and Rollback

This is one of the most important sections of the report.

Explain how inventory management was incorporated into the Order transaction. Describe how:

`product.stock -= item_data.quantity`
modifies inventory while the Order and OrderItems are being created.

Explain the roles of:

`db.commit()`
and

`db.rollback()`
in maintaining database consistency.

Successful transaction

Describe the successful Order transaction and show that the stock of the purchased products decreased after the transaction.

Include evidence showing inventory **before and after** the Order.

Failed transaction

Explain the experiment where an Order requested an unavailable quantity:

```
{
  "customer_id": 1,
  "items": [
    {"product_id": 1, "quantity": 1},
    {"product_id": 4, "quantity": 1000}
  ]
}
```

The expected failure is:

```
{
  "detail": "Insufficient stock for Webcam"
}
```

Explain why Product 1's stock must **not** decrease even though it was processed before the insufficient-stock condition for Product 4 was detected. Relate this behavior to **atomicity** and transaction rollback.

9. Independent Python Client and JSON

Describe the purpose of the separate Python client.

Explain how the client communicates with FastAPI using:

requests.get(...)
requests.post(...)
requests.put(...)
requests.delete(...)
and JSON messages.

Also explain how order.json is read using json.load() and subsequently transmitted to the /orders endpoint.

Evidence: Include a screenshot showing the Python client menu and the successful execution of at least one operation.

10. Client/Server Test

Briefly explain the optional experiment where the Python client and FastAPI application execute on different computers.

Explain the architecture:

```
Computer A                  Computer B

Python Client --- HTTP/JSON ---> FastAPI
                                   |
                                   | ORM
                                   v
                                 MySQL
```

Discuss what must be changed in BASE_URL when the client is moved to another computer and why FastAPI must listen on 0.0.0.0.

11. Final Exercise

Document your implementation of:

GET /products/{product_id}/customers
Explain how your SQLAlchemy query finds all customers who purchased a particular product.

Include:

1. The Python code implementing the endpoint and client. (github repository)
2. The result of:

curl -s http://127.0.0.1:8000/products/1/customers | python -m json.tool

3. A brief explanation of the .join() operations.

An explanation of why .distinct() is necessary.

The workshop provides the Customer → Order → OrderItem → Product relationship and a starting SQLAlchemy query for this exercise.

12. Discussion Questions

Answer the following questions in your own words:

1. What is an ORM, and what advantages does it provide compared with writing SQL directly?
2. What is the difference between a SQLAlchemy ORM model and a Pydantic schema in this application?
3. What is the purpose of `db.commit()`?
4. Why is `db.rollback()` important?
5. How does SQLAlchemy represent a one-to-many relationship?
6. What does `cascade="all, delete-orphan"` accomplish?
7. Why does deleting an Order remove its OrderItems but not its Products?
8. What advantage does a nested insert provide when creating an Order?
9. Why should an Order and its corresponding inventory modifications be executed as a single transaction?
10. What is the role of FastAPI in the architecture, and what is the role of SQLAlchemy?

13. Conclusions

Write **3–5 conclusions** based on the workshop. Do not simply state that the workshop was completed successfully.

Your conclusions should discuss what you learned about:

13. mapping Python objects to relational tables;
14. ORM relationships;
15. CRUD operations;
16. nested objects;
17. cascade operations;
18. transactions and rollback;
19. separation between the API, ORM/database layer, and external client.